

Cyclo: Lightweight Lattice-based Folding via Partial Range Checks

Garreta, Lipmaa, Luhaäär, Osadnik (2025)

1. Description

Cyclo is a new lattice-based folding scheme that improves upon LatticeFold+ in multiple dimensions. Key results:

- **Proof size:** ~30 KB (vs ~100 KB for LatticeFold+), an order-of-magnitude improvement.
- **Prover time:** ~3.5× faster than LatticeFold+ for the dominant commitment step.
- **Design:** An *amortized norm-refreshing* folding scheme — norm checks and decomposition are applied **only to the input relation** Ξ_0 , never to the accumulated relation Ξ_{acc} .
- **Folding depth:** Bounded (e.g., $\ell_{\text{fold}} \in [2^7, 2^{20}]$); can be refreshed by running LatticeFold+ every ℓ_{fold} steps.
- **For R1CS/CCS over $\mathbb{F}_q$:** No witness decomposition is needed inside the folding protocol itself, since witnesses are encoded as low-norm ring elements via $\theta_k : \mathcal{R}_q \rightarrow \mathbb{F}_q$.

2. Generic Idea of Lattice-based Folding

What is folding?

A **folding scheme** is a reduction of knowledge from $\Xi_{\text{acc}} \times \Xi_0$ to Ξ_{acc} . Given an accumulated instance and a fresh input instance, the prover folds them into a new accumulated instance. Repeating this aggregates many computation steps into a single accumulator.

The norm problem

Lattice-based folding uses **Ajtai commitments** $\mathbf{A}\mathbf{w} = \mathbf{y}$ in place of Pedersen commitments, requiring the witness $\mathbf{w}$ to be **short** ($\|\mathbf{w}\| \leq B$). Naive linear combination $\hat{\mathbf{w}} = \mathbf{w}_0 + c\mathbf{w}_1$ with a short challenge c keeps correctness, but:

- **Correctness direction:** $\|\hat{\mathbf{w}}\| \leq B\gamma_{\mathcal{D}}$ — norm grows multiplicatively.
- **Extraction direction:** The extractor can only extract $\tilde{\mathbf{w}}$ with a short denominator (slack factor), whose norm grows exponentially over rounds.

LatticeFold(+) framework

Both LatticeFold and LatticeFold+ address this with the following pipeline applied to **both** the input and accumulated relations:

$$\left. \begin{array}{l} (\Xi_{\text{acc}}^{\text{lin}})^\ell \xrightarrow{\text{norm-check}} (\Xi_{\text{acc}}^{\text{lin}'})^\ell \\ \Xi_{\text{input}}^{\text{lin}} \xrightarrow{\text{norm-check}} \Xi_{\text{input}}^{\text{lin}'} \end{array} \right\} \xrightarrow{\text{fold}} \Xi_{\text{folded}}^{\text{lin}} \xrightarrow{\text{decomp}} (\Xi_{\text{output}}^{\text{lin}})^\ell$$

- **Decomposition**: Split a witness of norm $\leq B$ into ℓ chunks of smaller norm b , so after random linear combination the norm stays bounded. Needed for completeness.
- **Range check**: Certify that a committed witness satisfies a norm bound. Needed for soundness (extraction).

Costs: LatticeFold uses k sum-checks (one per chunk); LatticeFold+ uses a lighter decomposition into 2 vectors but requires expensive "double commitments" ($\geq \varphi$ single Ajtai commitments).

3. High-Level Idea of Cyclo

Key observation

If we **skip** the norm check on the accumulated witness $\mathbf{v}$ (from Ξ_{acc}), the output accumulated witness is:

$$\hat{\mathbf{v}} = \mathbf{v} + \sum_{j \in [L]} s_j \mathbf{v}'_j$$

where $\mathbf{v}'_j$ are the (range-checked) extended input witnesses with $\|\mathbf{v}'_j\| \leq b$, and $\|s_j\|_{\text{op}} \leq \gamma$. Hence:

$$\|\hat{\mathbf{v}}\| \leq \|\mathbf{v}\| + L\gamma b$$

The norm grows **additively** by $L\gamma b$ per round (not multiplicatively). After ℓ_{fold} rounds, $\|\mathbf{v}\| \leq \beta + \ell_{\text{fold}} \cdot L\gamma b$, which remains acceptable.

New framework: Amortized Norm-Refreshing

The Cyclo pipeline is:

$$\Xi_{\text{input}}^{\text{lin}} \xrightarrow{\text{decomp}^*} \Xi_{\text{input}}^{\text{lin}'} \xrightarrow{\text{range-check}} \Xi_{\text{input}}^{\text{lin}''} \left. \begin{array}{l} \Xi_{\text{acc}}^{\text{lin}} \end{array} \right\} \xrightarrow{\text{fold}} \Xi_{\text{output}}^{\text{lin}}$$

The decomposition step (*) is **optional**: when folding R1CS/CCS over $\mathbb{F}_q$ with the θ_k encoding, the input witness already has norm $\leq b$, so it can be skipped entirely.

Two building blocks

1. **Extension Commitment** Π^{ext} : Decomposes the input witness $\mathbf{w} \in \mathcal{R}_q^m$ into $\ell = \lceil \log_{2b} 2B \rceil$ digits $\mathbf{w}_i$ (each with $\|\mathbf{w}_i\| \leq b$) and commits "vertically" (keeping it a single linear relation). Reduces the norm to b before the range check.

2. **Range Check** Π^{range} : Proves $\|\mathbf{w}\| \leq b$ using a sum-check over $\mathbb{F}_{q^e}$ on the polynomial $\prod_{j=-b}^b (f(X) - j)$ vanishing on the Boolean hypercube, where $f = \text{MLE}[\text{cf}(\mathbf{w})]$.

Efficiency gains

- Only **2 non-batched sum-checks** over $\mathbb{F}_{q^e}$ per fold (vs. batched sum-checks over k chunks in LatticeFold, or double commitments in LatticeFold+).
- No decomposition or range check on the accumulator.
- When R1CS/CCS over $\mathbb{F}_q$: extension commitment step is entirely skipped.

4. Background

Cyclotomic rings

$\mathcal{R} = \mathbb{Z}[X]/\langle \Phi_f(X) \rangle$, degree $\varphi = \varphi(f)$. $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$.

Norm: $\|\cdot\|$ denotes the ℓ_∞ norm on the coefficient vector. Operator norm:

$$\|c\|_{\text{op}} = \sup_{\mathbf{t} \neq 0} \|\mathbf{t} \cdot c\|_\infty / \|\mathbf{t}\|_\infty.$$

Coefficient embedding: $\text{cf} : \mathcal{R} \rightarrow \mathbb{Z}^\varphi$ (powerful basis). Dual basis $\text{cf}^\vee$ satisfies $\text{ct}(a \cdot b) = \langle \text{cf}(a), \text{cf}^\vee(b) \rangle$.

Recall the transform $\bar{a}(X) = a(X^{-1})$ in Neo/SuperNeo.

Principal Linear Relation (PLR)

$$\Xi_{\mathbf{A}, (\mathbf{M}_i)_{i \in [k], a, n, m, B}}^{\text{lin}} := \left\{ \begin{array}{l} ((\mathbf{r}_i)_{i \in [k]}, (\mathbf{b}_i)_{i \in [n]}, \mathbf{y}), \mathbf{w} : \\ (\mathbf{M}_i \in \mathcal{R}_q^{m_i \times m}, \mathbf{r}_i \in \mathcal{R}_q^{\log m_i})_{i \in [k]}, \quad (\mathbf{b}_i \in \mathcal{R}_q^{\log m})_{i \in [n]}, \\ \mathbf{A} \in \mathcal{R}_q^{a \times m}, \mathbf{y} = \begin{pmatrix} \bar{\mathbf{y}} \in \mathcal{R}_q^a \\ \mathbf{y} \in \mathcal{R}_q^{k+n} \end{pmatrix}, \mathbf{w} \in \mathcal{R}_q^m, \|\mathbf{w}\| \leq B \\ \left(\begin{array}{c} \mathbf{A} \\ \text{tensor}(\mathbf{r}_0)^T \mathbf{M}_0 \\ \vdots \\ \text{tensor}(\mathbf{r}_{k-1})^T \mathbf{M}_{k-1} \\ \text{tensor}(\mathbf{b}_0)^T \\ \vdots \\ \text{tensor}(\mathbf{b}_{n-1})^T \end{array} \right) \mathbf{w} = \mathbf{y} \bmod q \end{array} \right\}$$

The three constraint families:

- **(i) $\mathbf{A}$** : Random matrix. Reduces security to SIS hardness.
- **(ii) $(\mathbf{M}_i, \mathbf{r}_i)$** : Fixed matrices with batching vectors $\mathbf{r}_i$. Encodes R1CS/CCS constraints (output of sum-check linearization).
- **(iii) $\mathbf{b}_i$** : Evaluation claims output by the sum-check protocol.

5. Technical Details

5.1 Range Check Π_b^{range}

Goal: Given $\mathbf{w} \in \mathcal{R}_q^m$ committed in a PLR, prove $\|\mathbf{w}\| \leq b$.

Idea: $\|\mathbf{w}\| \leq b$ iff $g(X) = \prod_{j=-b}^b (f(X) - j)$ vanishes on $\{0, 1\}^\ell$, where $f = \text{MLE}[\text{cf}(\mathbf{w})]$ and $\ell = \lceil \log(m\varphi) \rceil$.

Protocol (over $\mathbb{F}_{q^e}$):

1. Verifier samples $\boldsymbol{\eta} \leftarrow \mathbb{F}_{q^e}^\ell$; set $\omega(X) = \text{eq}(X; \boldsymbol{\eta})$.
2. Set $\hat{f}(X) = \prod_{j=-b}^b (f(X) - j) \cdot \omega(X)$. Run sum-check to reduce $\sum_{\mathbf{z} \in \{0,1\}^\ell} \hat{f}(\mathbf{z}) \stackrel{?}{=} 0$ to a leaf check $\hat{f}(\mathbf{u}) \stackrel{?}{=} s$.
3. Prover sends $\tilde{t} = \langle \text{cf}_V^{-1}(\text{tensor}(\mathbf{u})), \mathbf{w} \rangle \in \mathcal{R}_{q^e}$. This is to describe the inner-product relation over rings.
4. Verifier computes $t = \text{Trace}(\tilde{t})$ and checks $s \stackrel{?}{=} \omega(\mathbf{u}) \cdot \prod_{j=-b}^b (t - j)$.
5. **Output:** PLR with one new row $\text{cf}_V^{-1}(\text{tensor}(\mathbf{u}))$ appended (an evaluation claim).

Communication: 1 element of $\mathcal{R}_{q^e}$ and $(2b + 2)\ell + 1$ elements of $\mathbb{F}_{q^e}$.

Range Test

$$\Pi_b^{\text{range}} : (((\mathbf{r}_i)_{i \in [k]}, (\mathbf{b}_i)_{i \in [n]}, \mathbf{y}), \mathbf{w}) \in \Xi_{n,b}^{\text{lin}} \rightarrow \Xi_{n+1,b}^{\text{lin}}$$

1. Prover defines $f := \text{MLE}[\text{cf}(\mathbf{w})] \in \mathbb{Z}_q[X_0, \dots, X_{\ell-1}]$ with $\ell = \lceil \log(m\varphi) \rceil$. Let $\text{tensor}(\cdot) \in \mathbb{F}_{q^e}^{m\varphi}$ denote the multilinear evaluation tensor such that $\langle \text{tensor}(\mathbf{z}), \text{cf}(\mathbf{w}) \rangle = \text{MLE}[\text{cf}(\mathbf{w})](\mathbf{z})$ for all $\mathbf{z} \in \mathbb{F}_{q^e}^\ell$. We view $\text{tensor} : \mathbb{F}_{q^e}^\ell \rightarrow \mathbb{F}_{q^e}^{m\varphi}$ as the standard Boolean-ML extension tensor.
2. The verifier samples $\boldsymbol{\eta} = (\eta_0, \eta_1, \dots, \eta_{\ell-1}) \leftarrow \mathbb{F}_{q^e}$ and sets $\omega(X) := \text{eq}(X; \boldsymbol{\eta})$.
3. Prover sets $\hat{f}(X) := \prod_{j=-b}^b (f(X) - j) \cdot \omega(X) \in \mathbb{F}_{q^e}[X_0, \dots, X_{\ell-1}]$.
4. The parties run sum-check over $\mathbb{F}_{q^e}$, reducing $\sum_{\mathbf{z} \in \{0,1\}^\ell} \hat{f}(\mathbf{z}) \stackrel{?}{=} 0$ to $\hat{f}(\mathbf{u}) \stackrel{?}{=} s$, where $\mathbf{u}$ is the challenge vector sampled by the verifier and $s \in \mathbb{F}_{q^e}$ is the claimed leaf value sent by the prover.
5. Prover sets $\tilde{t} := \langle \mathbf{u}', \mathbf{w} \rangle \in \mathcal{R}_{q^e}$ for $\mathbf{u}' := \text{cf}_V^{-1}(\text{tensor}(\mathbf{u}))$ and sends it to the verifier (here $\text{cf}_V^{-1}(\text{tensor}(\mathbf{u})) \in \mathcal{R}_{q^e}^m$ so the inner product lies in $\mathcal{R}_{q^e}$).
6. Verifier computes $t := \text{Trace}(\tilde{t}) \in \mathbb{F}_{q^e}$ and checks $s \stackrel{?}{=} \omega(\mathbf{u}) \cdot \prod_{j=-b}^b (t - j)$. By Lemma 1 (dual-basis trace), $\text{Trace}(\langle \text{cf}_V^{-1}(\text{tensor}(\mathbf{u})), \mathbf{w} \rangle) = \text{MLE}[\text{cf}(\mathbf{w})](\mathbf{u})$. Since $\text{Trace}(\tilde{t}) = \text{MLE}[\text{cf}(\mathbf{w})](\mathbf{u}) = f(\mathbf{u})$, the leaf check equals $\hat{f}(\mathbf{u}) \stackrel{?}{=} s$.
7. Prover and verifier output: $(((\mathbf{r}_i)_{i \in [k]}, ((\mathbf{b}_i)_{i \in [n]}, \mathbf{u}'), (\frac{\mathbf{y}}{t})), \mathbf{w}) \in \Xi_{n+1,b}^{\text{lin}}$.

5.2 Extension Commitment $\Pi_{b,\mathcal{C}}^{\text{ext}}$

Goal: Reduce a PLR with large-norm witness ($\|\mathbf{w}\| \leq B$) to one with small-norm witness ($\|\mathbf{v}\| \leq b$), via a "vertical" decomposition.

Protocol:

1. Prover splits $\mathbf{w} = \sum_{i=0}^{\ell-1} \mathbf{w}_i \cdot (2b)^i$ with $\|\mathbf{w}_i\| \leq b$. Let $\mathbf{v}^T = (\mathbf{w}_0^T, \dots, \mathbf{w}_{\ell-1}^T) \in \mathcal{R}_{q^e}^{m\ell}$.

2. Prover sends commitment $\mathbf{t} = \mathbf{R}\mathbf{v} \in \mathcal{R}_q^{a'}$.
3. Verifier samples $\hat{\mathbf{c}} \leftarrow \mathcal{C}^{\ell_C}$; set $\mathbf{c} = \text{tensor}(\hat{\mathbf{c}}) \in \mathcal{R}_{q^e}^a$.
4. Prover and verifier add the row $\mathbf{f}^T = \mathbf{c}^T \cdot ((2b)^0, \dots, (2b)^{\ell-1}) \otimes \mathbf{A}$, certifying $\mathbf{f}^T \mathbf{v} = \langle \mathbf{c}, \mathbf{y} \rangle$ (i.e., $\mathbf{v}$ is a valid decomposition of a witness for the original constraint).
5. **Output:** new PLR over $\mathbf{R}$ with witness $\mathbf{v}$, norm $\leq b$.

Key point: Decomposition is "vertical" — $\mathbf{v}$ is still a single long witness for a single linear relation (vs. LatticeFold's "horizontal" split into k separate relations). This saves communication.

Communication: a' elements of $\mathcal{R}_q$.

Extension commitment

$$\Pi_{b,\mathcal{C}}^{\text{ext}} : (((\mathbf{r}_i)_{i \in [k]}, (\mathbf{b}_i)_{i \in [n]}, \mathbf{y}), \mathbf{w}) \in \Xi_{\mathbf{A}, (\mathbf{M}_i)_{i \in [k]}, a, n, m, B}^{\text{lin}} \rightarrow \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}, a', n, m\ell, b}^{\text{lin}}$$

1. The prover splits the witness: $\mathbf{v}^T = (\mathbf{w}_0^T, \dots, \mathbf{w}_{\ell-1}^T)$, where $\mathbf{w}_i$ are such that $\|\mathbf{w}_i\|_\infty \leq b$ for all $i \in [\ell]$ and $\mathbf{w} = \sum_{i=0}^{\ell-1} \mathbf{w}_i b^i$.
2. The prover sends the extended commitment $\mathbf{t} := \mathbf{R}\mathbf{v} \in \mathcal{R}_q^{a'}$ to the verifier.
3. The verifier samples $\hat{\mathbf{c}} \leftarrow \mathcal{C}^{\ell_C}$ and sets $\mathbf{c} := \text{tensor}(\hat{\mathbf{c}}) \in \mathcal{R}_{q^e}^a$.
4. Let
 - (i) $\widehat{\mathbf{b}}_i^T := (((2b)^0, (2b)^1, \dots, (2b)^{\ell-1}) \otimes \mathbf{b}_i^T) \in \mathcal{R}_{q^e}^{[\log m] + \ell} \quad \forall i \in [k]$,
 - (ii) $\widehat{\mathbf{M}}_i := ((2b)^0, (2b)^1, \dots, (2b)^{\ell-1}) \otimes \mathbf{M}_i \in \mathcal{R}_q^{m_i \times m\ell} \quad \forall i \in [k]$,
 - (iii) $\widehat{\mathbf{M}}_k := ((2b)^0, (2b)^1, \dots, (2b)^{\ell-1}) \otimes \mathbf{A} \in \mathcal{R}_q^{a \times m\ell}$,
 The prover and verifier output the augmented instance

$$(((\mathbf{r}_i)_{i \in [k]}, \mathbf{c}), (\widehat{\mathbf{b}}_i)_{i \in [n]}, \widehat{\mathbf{y}}), \mathbf{v}) \in \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}, a', n, m\ell, b}^{\text{lin}}$$

$$\text{for } \widehat{\mathbf{y}}^T := (\mathbf{t}^T, y_a, \dots, y_{a+k-1}, \langle \mathbf{c}, (y_0, \dots, y_{a-1}) \rangle, y_{a+k}, \dots, y_{a+k+n-1}).$$

5.3 Folding Scheme $\Pi_{b,\mathcal{D},\mathcal{C}}^{\text{fs}}$

Input: Accumulated instance

$$(\mathbf{x}_{\text{acc}}, \mathbf{v}) \in \Xi_{\text{acc}, \beta}^{\text{lin}}$$

and L input instances $(\mathbf{x}'_j, \mathbf{w}'_j) \in \Xi_{a, n, m, B}^{\text{lin}}$.

Steps:

1. **Extension commitment** on each input: $(\mathbf{x}''_j, \mathbf{v}'_j) \leftarrow \Pi^{\text{ext}}(\mathbf{x}'_j, \mathbf{w}'_j)$.
2. **Range check** on each extended input: prove $\|\mathbf{v}'_j\| \leq b$.
3. **Unification/Linear check:** Batch all sum-check claims (accumulator + inputs, below (a) (b)(c)(d)) using shared random $\mathbf{d}$, reducing to evaluation claims at a shared random point $\hat{\mathbf{r}}$.
4. **Linear combination:** Verifier sends short challenge $\mathbf{s} \leftarrow \mathcal{D}^L$. Output:

$$\hat{\mathbf{v}} = \mathbf{v} + \sum_{j \in [L]} s_j \mathbf{v}'_j \in \Xi_{\text{acc}, \beta + Lb\gamma}^{\text{lin}}$$

Norm analysis:

- **Correctness:** $\|\hat{\mathbf{v}}\| \leq \|\mathbf{v}\| + L\gamma b = \beta + Lb\gamma$.
- **Soundness:** Coordinate-wise forking extracts $\mathbf{v}'_j$ (via Π^{range} extractor) and reconstructs the accumulated witness as $\mathbf{v}^* = \hat{\mathbf{v}} - \sum_j s_j \mathbf{v}'_j$.

Folding Scheme

$$\Pi_{b,\mathcal{D},\mathcal{C}}^{\text{fs}} : \left(\begin{array}{l} ((\mathbf{r}_i)_{i \in [k]}, \mathbf{b}, \mathbf{y}), \mathbf{v} \in \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}, a', 1, m \log_{2b} 2B, \beta}^{\text{lin}} \\ ((\mathbf{r}'_{i,j})_{i \in [k]}, (\mathbf{b}'_{i,j})_{i \in [n]}, \mathbf{y}'_j), \mathbf{w}'_j \in (\Xi_{\mathbf{A}, (\mathbf{M}_i)_{i \in [k]}, a', n, m, B}^{\text{lin}})^L \\ \rightarrow \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}, a', 1, m \log_{2b} 2B, \beta + Lb\gamma}^{\text{lin}} \end{array} \right)$$

1. Prover and verifier compute for all $j \in [L]$: // see Fig. 2

$$((\mathbf{r}''_{i,j})_{i \in [k+1]}, (\mathbf{b}'_{i,j})_{i \in [n]}, \mathbf{t}'_j, \mathbf{v}'_j) \leftarrow \Pi_{b,\mathcal{C}}^{\text{ext}}(((\mathbf{r}'_{i,j})_{i \in [k]}, (\mathbf{b}'_{i,j})_{i \in [n]}, \mathbf{y}'_j), \mathbf{w}'_j).$$

2. Prover and verifier compute for all $j \in [L]$: // see Fig. 1

$$((\mathbf{r}''_{i,j})_{i \in [k+1]}, (\mathbf{b}''_{i,j})_{i \in [n+1]}, \mathbf{t}'_j, \mathbf{v}'_j) \leftarrow \Pi_b^{\text{range}}(((\mathbf{r}''_{i,j})_{i \in [k+1]}, (\mathbf{b}'_{i,j})_{i \in [n]}, \mathbf{t}'_j), \mathbf{v}'_j).$$

3. Verifier samples $\mathbf{d} \leftarrow \mathbb{F}_{q^e}^{\lceil \log \varphi(2+k+L(2+n+k)) \rceil}$.

4. Let $\tilde{m} := m \log_{2b} 2B$ and $\tilde{m}_i := m_i \log_{2b} 2B$ for $i \in [k+1]$. Prover and verifier express linear equalities concerning input relations as sum-check instances:

- (a) $\sum_{\mathbf{z} \in \{0,1\}^{\log \tilde{m}_i}} \text{MLE}[\widehat{\mathbf{M}}_i \mathbf{v}'_j](\mathbf{z}) \text{eq}(\mathbf{z}; \mathbf{r}''_{i,j}) = t'_{j,a'+i}, \quad i \in [k+1] \ j \in [L],$
- (b) $\sum_{\mathbf{z} \in \{0,1\}^{\log \tilde{m}_i}} \text{MLE}[\widehat{\mathbf{M}}_i \mathbf{v}](\mathbf{z}) \text{eq}(\mathbf{z}; \mathbf{r}_i) = y_{a'+i}, \quad i \in [k+1],$
- (c) $\sum_{\mathbf{z} \in \{0,1\}^{\log \tilde{m}}} \text{MLE}[\mathbf{v}'_j](\mathbf{z}) \text{eq}(\mathbf{z}; \mathbf{b}'_{i,j}) = t'_{j,a'+k+1+i}, \quad i \in [n+1] \ j \in [L],$
- (d) $\sum_{\mathbf{z} \in \{0,1\}^{\log \tilde{m}}} \text{MLE}[\mathbf{v}](\mathbf{z}) \text{eq}(\mathbf{z}; \mathbf{b}) = y_{a'+k+1},$

Then, they batch the sum-check claims^a with $\mathbf{d}$ and execute the sum-check protocol (over $\mathbb{F}_{q^e}$ NTT slots of $\mathcal{R}_{q^e}$) to reduce them to claims over a shared random point, i.e., $((\widehat{\mathbf{r}}_i)_{i \in [k]}, \widehat{\mathbf{r}}, \widehat{\mathbf{y}}), \mathbf{v} \in \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}}^{\text{lin}}$ and

$$((\widehat{\mathbf{r}}_i)_{i \in [k]}, \widehat{\mathbf{r}}, \widehat{\mathbf{y}}_j, \mathbf{v}'_j) \in \Xi_{\mathbf{A}, (\mathbf{M}_i)_{i \in [k]}}^{\text{lin}} \text{ for } j \in [L].$$

The new evaluation claims $\widehat{\mathbf{y}}[a', |\widehat{\mathbf{y}}|]$ and $\widehat{\mathbf{y}}'_j[a', |\widehat{\mathbf{y}}'_j|]$ (over $\mathcal{R}_{q^e}$) are sent to the verifier.

5. Verifier sends a short folding challenge $\mathbf{s} \leftarrow \mathcal{D}^L$.
6. Prover and verifier output:

$$\left(\left((\widehat{\mathbf{r}}_i)_{i \in [k]}, \widehat{\mathbf{r}}, \widehat{\mathbf{y}} + \sum_{j \in [L]} s_j \widehat{\mathbf{y}}'_j \right), \mathbf{v} + \sum_{j \in [L]} s_j \mathbf{v}'_j \right) \in \Xi_{\mathbf{R}, (\widehat{\mathbf{M}}_i)_{i \in [k+1]}, a', 1, \tilde{m}, \beta + Lb\gamma}^{\text{lin}}$$

where γ is the expansion factor of the challenge set $\mathcal{D}$.

^a Batching sum-check claims with different numbers of variables requires padding functions with fewer variables using unused auxiliary variables and later erasing the corresponding coordinates from the output evaluation point.

5.4 R1CS over $\mathbb{F}_q$ to PLR

1. Module homomorphism θ_k

For a positive integer k , define:

$$\theta_k : \mathcal{R}_q \rightarrow \mathbb{F}_q, \quad f(X) \mapsto f(k) \bmod q$$

This is an $\mathbb{F}_q$ -module homomorphism (not a ring homomorphism in general). Key property:

$$\theta_k(\text{MLE}[\mathbf{v}](\mathbf{x})) = \text{MLE}[\theta_k(\mathbf{v})](\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{F}_q^{\log m}$$

2. Reduction

Encoding: Given $c = \sum c_i k^i \in \mathbb{F}_q$ with base- k digits $c_0, \dots, c_{\ell_k(q)-1} \in [0, k-1]$, define $p_c(X) = \sum_i c_i X^i \in \mathcal{R}_q$. Then $\theta_k(p_c(X)) = c$ and $\|p_c\|_\infty < k$.

Goal: Reduce $(\mathbf{M}_0 \cdot \mathbf{w}) \circ (\mathbf{M}_1 \cdot \mathbf{w}) = \mathbf{M}_2 \cdot \mathbf{w}$ over $\mathbb{F}_q$ to the PLR over $\mathcal{R}_q$.

Step 1 — Encode (Committed Hybrid R1CS): For each $w_i \in \mathbb{F}_q$, set

$w'_i = p_{w_i}(X) = \sum_j (w_i)_j X^j \in \mathcal{R}_q$ (base- k digit polynomial). Then $\theta_k(w'_i) = w_i$ and $\|w'_i\|_\infty \leq k$. Commit $\mathbf{A}\mathbf{w}' = \mathbf{y}$. The R1CS constraint becomes:

$$(\mathbf{M}_0 \cdot \theta_k(\mathbf{w}')) \circ (\mathbf{M}_1 \cdot \theta_k(\mathbf{w}')) - \mathbf{M}_2 \cdot \theta_k(\mathbf{w}') = \mathbf{0}$$

Step 2 — Linearize (following HyperNova): Run sum-check over $\mathbb{F}_{q^e}$ to reduce the degree-2 constraint. Define:

$$Q_i(\mathbf{Y}) = \sum_{\mathbf{b}' \in \{0,1\}^{\log m}} \text{MLE}[\mathbf{M}_i](\mathbf{Y}, \mathbf{b}') \cdot \text{MLE}[\theta_k(\mathbf{w}')](\mathbf{b}')$$

Sum-check reduces $\sum_{\mathbf{b}} Q(\mathbf{b})\text{eq}(\mathbf{b}; \mathbf{r}) = 0$ to a claim at random $\mathbf{u}$.

Step 3 — Lift to $\mathcal{R}_q$: By Lemma 6 (θ_k commutes with MLE), the prover publishes $d'_i \in \mathcal{R}_{q^e}$ with $\theta_k(d'_i) = d_i$ instead of $d_i \in \mathbb{F}_{q^e}$.

Key advantage: Sum-check runs over $\mathbb{F}_{q^e}$ (cheap), not over $\mathcal{R}_{q^e}$ (expensive). The θ_k map bridges the two structures, providing a simpler algebraic framework than Neo's matrix-algebraic machinery.

Skipping extension commitment: If $k \leq b$, the encoded witness already has norm $\leq b$, so Π^{ext} is skipped in the folding step.

From committed hybrid R1CS to the principal linear relation

$$\Pi^{\text{hyb-R1CS}} : \left(((\mathbf{x} \in \mathcal{R}_q^\ell, \mathbf{y} \in \mathcal{R}_q^a, \mathbf{w} \in \mathcal{R}_q^{m-\ell-1}) \in \Xi_{\mathcal{R}_q, \theta_k, \mathbf{A}, (\mathbf{M}_i)_{i \in [3]}, a, m, \ell, B}^{\text{com-hyb-R1CS}} \right) \rightarrow ((\mathbf{r}' \in \mathcal{R}_{q^e}^{\log(m)})^3, \mathbf{b}' \in \mathcal{R}_{q^e}^{\log m}, \mathbf{y}' \in \mathcal{R}_q^a \times \mathcal{R}_{q^e}^4, \mathbf{w}' \in \mathcal{R}_q^m) \in \Xi_{\mathbf{A}, (\mathbf{M}_i)_{i \in [3]}, a, 1, m, B}^{\text{lin}}$$

1. Verifier samples a random vector $\mathbf{r} \in \mathbb{F}_{q^e}^{\log m}$ and sends $\mathbf{r}$ to the prover.
2. Define $\mathbf{w}' = (\mathbf{x}, 1, \mathbf{w}) \in \mathcal{R}_q^m$, let $\theta_k(\mathbf{w}') \in \mathbb{F}_q^m$ be the result of applying θ_k to $\mathbf{w}'$ component-wise, and let $Q(\mathbf{Y})$ be the polynomial on variables $\mathbf{Y} = (Y_0, \dots, Y_{\log m-1})$ and coefficients in $\mathbb{F}_q$ defined as

$$Q(\mathbf{Y}) = Q_0(\mathbf{Y})Q_1(\mathbf{Y}) - Q_2(\mathbf{Y})$$

$$Q_i(\mathbf{Y}) = \sum_{\mathbf{b}' \in \{0,1\}^{\log m}} \text{MLE}[\mathbf{M}_i](\mathbf{Y}, \mathbf{b}') \text{MLE}[\theta_k(\mathbf{w}')](\mathbf{b}') \quad \text{for } i \in [3].$$

3. Prover and verifier engage in the sum-check protocol over $\mathbb{F}_{q^e}$ and reduce

$$\sum_{\mathbf{b} \in \{0,1\}^{\log m}} Q(\mathbf{b})\text{eq}(\mathbf{b}; \mathbf{r}) \stackrel{?}{=} 0 \rightarrow Q(\mathbf{u})\text{eq}(\mathbf{u}, \mathbf{r}) \stackrel{?}{=} c,$$

where $\mathbf{u} \in \mathbb{F}_{q^e}^{\log m}$ is the vector of challenges sampled by the verifier, and $c \in \mathbb{F}_{q^e}$ is certain field element determined during the execution of the sum-check protocol.

4. Prover computes the ring elements $d_i = \sum_{\mathbf{b}' \in \{0,1\}^{\log m}} \text{MLE}[\mathbf{M}_i](\mathbf{u}, \mathbf{b}') \text{MLE}[\mathbf{w}'](\mathbf{b}') \in \mathcal{R}_{q^e}$ for $i \in [3]$, and sends these to the verifier.
5. Verifier asserts $(\theta_k(d_0)\theta_k(d_1) - \theta_k(d_2))\text{eq}(\mathbf{u}; \mathbf{r}) \stackrel{?}{=} c$.
6. Verifier samples $\mathbf{v} \in \mathcal{C}^{\log(\ell)+1}$ and sends $\mathbf{v}$ to prover. This will be used as an evaluation point to guarantee that the output witness $\mathbf{w}'$ starts with the public input $(\mathbf{x}, 1)$.
7. Prover and verifier compute the ring element $e = \text{MLE}[(\mathbf{x}, 1)](\mathbf{v}) \in \mathcal{R}_{q^e}$, and output $((\mathbf{r}'_i)_{i \in [3]}, \mathbf{b}', \mathbf{y}'), \mathbf{w}'$, where:

$$\mathbf{r}' = (\mathbf{u}, \mathbf{u}, \mathbf{u}) \in \mathcal{R}_{q^e}^{3 \times \log(m)} \quad \mathbf{b}' = (\mathbf{v}, \mathbf{0}) \in \mathcal{R}_{q^e}^{\log m}$$

$$\mathbf{y}' = (\mathbf{y}, d_0, d_1, d_2, e) \in \mathcal{R}_q^a \times \mathcal{R}_{q^e}^4 \quad \mathbf{w}' = (\mathbf{x}, 1, \mathbf{w}) \in \mathcal{R}_q^m$$